

PROGRAMMING FOR DATA SCIENCE

Chamika Deshan

Index: COMScDS251P-015

1. Setup

1.1 Virtual Environment

For the project we use Python for programming, and Python is managed by a virtual environment for shared for both programming questions. This is useful for keeping libraries organized and avoiding conflicts, so the project can run smoothly on different systems.

```
python3 -m venv .venv
source ./venv/bin/activate
```

1.2 Version Controlling

Git has been used for version controlling and can be find [here](#).

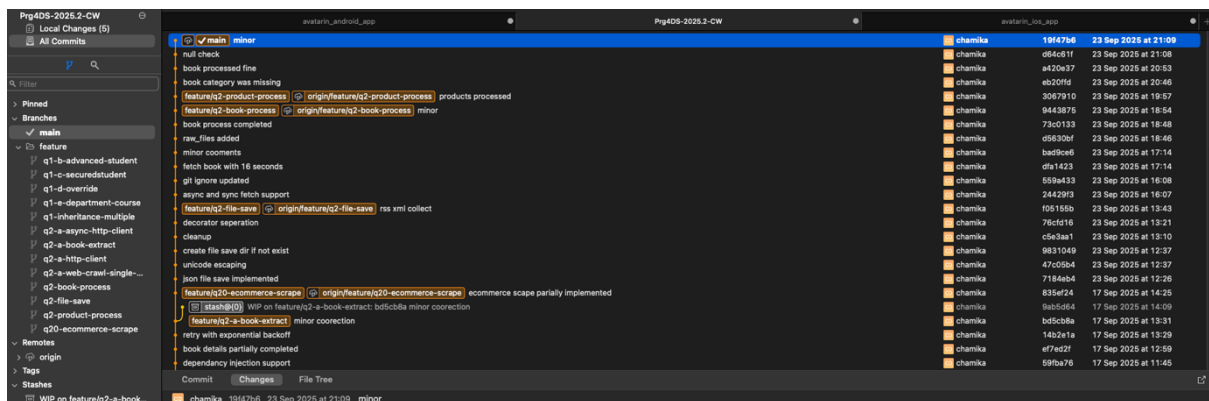


Figure 1: Github repo commits

1.3 Dependencies

Following is the list of dependencies for the project and can be installed to virtual environment with

```
pip install -r requirements.txt
```

```
beautifulsoup4==4.13.5
aiohttp==3.12.15
dependency-injector==4.48.1
aiofiles==24.1.0
lxml==6.0.2
requests==2.32.5
pandas==2.3.2
matplotlib==3.10.6
seaborn==0.13.2
numpy==2.3.3
plotly==6.3.0
```

Figure 2: requirements.txt

Two different libraries have been installed for HTTP requests: aiohttp for asynchronous coroutine support, and requests which is thread blocking (synchronous) calls.

2. Question 1: University Management System

2.1 Inheritance with Multiple Classes

Here “Person” class is selected as an abstract Base class which multiple other child classes such as “Student”, “Faculty” & “Staff” have their individual operations but all inheriting some common features from “Person” super class. The master class “Person” has common properties like person id and name. The derived class's constructor takes the “Person” class's parameters as inputs plus whatever else its own design requires. And they invoke the based class constructor as required.

```
from abc import ABC, abstractmethod

class Person(ABC):
    def __init__(self, person_id, name):
        self.person_id = person_id
        self.name = name

    @abstractmethod
    def get_responsibilities(self):
        pass
```

Figure 3: Base class “Person”

```
from person import Person
from model.academic_status import AcademicStatus
from model.results import Results
from manager.istudent_manager import IStudentManager

class Student(Person):
    def __init__(self, person_id, name, student_id, student_manager):
        super().__init__(person_id, name)
        self.student_id = student_id
        self._semester_courses = {}
        self._gpa = 0.0
        self._academic_status: AcademicStatus | None = None
        self._student_manager: IStudentManager | None = student_manager
```

Figure 4: Child class “Student”

```
from person import Person

# Faculty
class Faculty(Person):
    def __init__(self, person_id, name, department):
        super().__init__(person_id, name)
        self.department = department

    # override
    def get_responsibilities(self):
        return "Responsibilities of Faculty goes here"

    def calculate_workload(self):
        return 10
```

Figure 5: Child class “Faculty”

2.2 Advanced Student Management

2.2.1 Course enrollment system

For academic Status tracking created a enum class which allows better scalability and clear separation of concerns.

```

from enum import Enum

class AcademicStatus(Enum):
    GOOD = "Good Standing"
    PROBATION = "Probation"
    DEANS_LIST = "Deans List"

    @staticmethod
    def from_gpa(gpa: float):
        if gpa >= 3.5:
            return AcademicStatus.DEANS_LIST
        elif gpa < 2.0:
            return AcademicStatus.PROBATION
        else:
            return AcademicStatus.GOOD

```

Figure 5: AcedemicStatus enum class

To implement methods `enroll_course()`, `drop_course()`, `calculate_gpa()`, and `get_academic_status()`, a separate interface `IStudentManager` was used. This abstracts the student management logic and is implemented by `StudentManagerImp`. With this solution, the `Student` class is cleaner and focuses on its data attributes, while enrollment, course drops, GPA calculation, and result assignment are handled by the manager. This provides a clear separation of concerns and uses a composition relationship with the manager.

```

from manager.student_manager import IStudentManager

class Student(Person):
    def __init__(self, person_id, name, student_manager):
        super().__init__(person_id, name)
        self.gpa = 0.0
        self.academic_status: AcademicStatus | None = None
        self.student_manager: IStudentManager | None = student_manager
        self.course_results = []

    def enroll_course(self, semester, course):
        self.student_manager.enroll(self, semester, course)

    def drop_course(self, semester, course_code):
        self.student_manager.drop(self, semester, course_code)

    def calculate_gpa(self) -> float:
        self.gpa = self.student_manager.gpa(self)
        self.academic_status = AcademicStatus.from_gpa(self.gpa)
        return self.gpa

    def get_academic_status(self) -> AcademicStatus:
        return self.academic_status

```

Figure 6: "Student" class **has(composition)** student_manager to perform operations

```

class IStudentManager(ABC):
    """Interface for student management operations."""

    @abstractmethod
    def enroll(self, student, semester, course):
        """
        Enroll a student in a specific course for a given semester.

        Args:
            student: The student object to enroll.
            semester: The semester in which the student is enrolling.
            course: The course object in which the student is enrolling.
        """
        pass

    @abstractmethod
    def drop(self, semester, course_code):
        pass

    @abstractmethod
    def gpa(self, student) -> float:
        pass

```

Figure 7: Interface IStudentManager

The `Course` class was created to make the implementation more future proof. This helps by avoiding problems with method calls or unit tests and makes it easier to scale in the future. Without this method signature can become more complex and each time signature changes implementation of the invocation and unit tests also need to update significantly.

```

class Course:
    def __init__(self, code,
                 name,
                 credits = 3,
                 max_students_allowd = 10,
                 should_have_completed: List[str] = None,
                 course_manager: ICourseManager | None = None):
        self.course_code = code
        self.course_name = name
        self.course_credits = credits
        self.max_students_allowd = max_students_allowd
        self.reading_students: List[str] = []

```

Figure 8: Course class

According to the requirements, a student should have courses assigned to a semester. Each semester can have multiple courses. Using a dictionary is a good way to represent this relationship. And assignment can be done accordingly.

```
student.semester_courses[semester] = {}
student.semester_courses[semester][course.course_code] = course
```

To remove course we can just remove course from semester_course dictionary

```
del student.semester_courses[semester][course_code]
```

To calculate GPA, we need an additional method in the “Student” class to set results for a course. This method is also comply with interfacer/implementation and logics implemented at StudentManagerImp class. Its inputs are the results for a specific course and semester. Two additional classes, Grade and Results, were added to keep things separated. The Grade class can be implemented as an Enum for clearly define all possible grades a student can receive.

```
from enum import Enum

class Grade(Enum):
    A = 4.0
    A_MINUS = 3.7
    B_PLUS = 3.3
    B = 3.0
    B_MINUS = 2.7
    C_PLUS = 2.3
    C = 2.0
    C_MINUS = 1.7
    D = 1.0
    F = 0.0
```

Figure 9: Grade Enum class

```
class Results:
    def __init__(self, semester, course_code, grade):
        self.semester = semester
        self.course_code = course_code
        self.course_grade = grade
```

Figure 10: Results class

```
class Student(Person):
    def set_results(self, results: Results):
        self._student_manager.set_results(self, results)
```

Figure 11: Assign results for course

```
class IStudentManager(ABC):
    @abstractmethod
    def set_results(self, results: Results):
        pass
```

Figure 12: Assign results signature in interface

2.3 Encapsulation with Validation

Encapsulation with the Student, Results, and Course classes has been discussed above.

For the New class SecureStudentRecord attributes Students and gpa are identified as private properties. And appropriate Setters and Getters implemented.

```

class SecureStudentRecord:
    def __init__(self, student: Student):
        self._reset(student)

    def _reset(self, student: Student):
        self.__student = student
        self.__gpa: float | None = None

    # Setter/getter for student
    @property
    def student(self) -> Student:
        return self.__student

    @student.setter
    def student(self, new__student: Student):
        if not isinstance(new__student, Student):
            raise TypeError("Assigned value must be a Student instance.")
        self._reset(new__student)

    # gpa getter
    @property
    def gpa(self) -> float:
        if self.__gpa is None:
            self.__gpa = self.__student.calculate_gpa()
        return self.__gpa

    # allow set gpa from outside
    @gpa.setter
    def gpa(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("GPA must be a number.")
        if not (0.0 <= value <= 4.0):
            raise ValueError("GPA must be between 0.0 and 4.0.")

```

Figure 13: SecureStudentRecord class with getters and Setters

Validation is implemented in StudentManagerImp, which raises appropriate error messages when requirements are not met. The following are the key places where errors are thrown as needed.

1. When enrolling, check if the course is already assigned for the particular semester.
2. When dropping, check if the course exists in the particular semester.
3. When setting results, check both that the course exists and that results have not already been assigned.
4. When calculating GPA, ensure that results have been assigned.

```

class StudentManagerImp(IStudentManager):

    def enroll(self, student, semester, course):
        if course.course_code in student.semester_courses[semester]:
            raise ValueError(f"{student.name}, already enrolled in Course:{course.course_code}-{course.course_name}")

    def drop(self, student, semester, course_code):
        if semester not in student.semester_courses or course_code not in student.semester_courses[semester]:
            raise ValueError(f"{course_code}\t not found in {semester} courses for {student.name}")

    def set_results(self, student, result: Results):
        if result.course_code not in student.semester_courses[result.semester]:
            raise ValueError(
                f"{result.course_code} not found in {result.semester} courses for {student.name}"
            )

    def gpa(self, student) -> float:
        total_credits = 0.0
        if total_credits == 0:
            raise ValueError(f"No graded courses found for {student.name}. GPA cannot be calculated")

```

Figure 14: Course/Results validations

2.4 Polymorphism with Method Overriding

Person class has an abstract method get_responsibilities and this has been overridden in child classes.

```

from abc import ABC, abstractmethod

class Person(ABC):
    def __init__(self, person_id, name):
        self.person_id = person_id
        self.name = name

    @abstractmethod
    def get_responsibilities(self):
        pass

```

Figure 15: Person class with get_responsibilities

```

class Student(Person):
    def set_results(self, results: Results):
        self._student_manager.set_results(self, results)

    # override
    def get_responsibilities(self):
        return f"Responsibilities of Student[{self.name}] goes here"

class UndergraduateStudent(Student):
    def __init__(self, person_id, name, student_id, student_manager):
        super().__init__(person_id, name, student_id, student_manager)

    # override
    def get_responsibilities(self):
        return f"Responsibilities of UndergraduateStudent[{self.name}] goes here"

class GraduateStudent(Student):
    def __init__(self, person_id, name, student_id, student_manager):
        super().__init__(person_id, name, student_id, student_manager)

    # override
    def get_responsibilities(self):
        return f"Responsibilities of GraduateStudent[{self.name}] goes here"

```

Figure 16: Student, UndergraduateStudent, GraduateStudent overrides get_responsibilities

Faculty class has a method calculate_workload and this has been overridden in child classes with summing up with base class workload.

```

9 # Faculty
10 class Faculty(Person):
11     def calculate_workload(self):
12         return 10
13
14 # Professor
15 class Professor(Faculty):
16     # override
17     def calculate_workload(self):
18         return super().calculate_workload() + 50
19
20 # Lecturer
21 class Lecturer(Faculty):
22     # override
23     def calculate_workload(self):
24         return super().calculate_workload() + 40
25
26 # Teaching Assitant
27 class TA(Faculty):
28     def calculate_workload(self):
29         return super().calculate_workload() + 30

```

Figure 17: calcaulte_workload method overridden

2.5 Department and Course Management

The “Department” class has been implemented with the attributes [name, faculty memembers, courses and students]. The “Course” class described above has been extended with attributes [max students allowed, list of courses should have completed]. A Course Manager interface and implementation seperated to keep the design cleaner and to maintain seperation of concerns principles. CouseManager interface is injected to Course class at constructor level.

```

class Department:
    def __init__(self, name):
        self.name = name
        self.faculty_members = []
        self.courses = []
        self.students = []

    def add_faculty(self, faculty):
        if faculty in self.faculty_members:
            raise ValueError(f"{faculty.name} already in the department [{self.name}]")
        self.faculty_members.append(faculty)

    def add_course(self, course):
        if course in self.courses:
            raise ValueError(f"{course.course_code} already in the department [{self.name}] course list")
        self.courses.append(course)

    def add_student(self, student):
        if student in self.students:
            raise ValueError(f"{student.name} already in the department [{self.name}] student list")
        self.students.append(student)

```

Figure 18: Department class

```

class Course:
    def __init__(self, code, name, credits=3, max_students_allowed=10, should_have_completed: List[str] = None, course_manager: ICourseManager = None):
        self.course_code = code
        self.course_name = name
        self.course_credits = credits
        self.max_students_allowed = max_students_allowed
        self.reading_students: List[str] = []
        self.should_have_completed = should_have_completed
        self.course_manager: ICourseManager | None = course_manager
        self.assigned_faculty: List[str] = []

    def add_student(self, person_id: str, completed_courses: List[str]):
        self.course_manager.add_student(self, person_id, completed_courses)

    def remove_student(self, person_id: str):
        self.course_manager.remove_student(self, person_id)

    def add_faculty(self, faculty: Faculty):
        self.course_manager.assign_faculty(self, faculty)

```

```

# Course Logic interface, to separate student logics from Student model class
class ICourseManager(ABC):
    @abstractmethod
    def add_student(self, course, person_id: str, completed_courses: List[str]):
        pass

    @abstractmethod
    def remove_student(self, course, person_id: str):
        pass

    @abstractmethod
    def assign_faculty(self, course, faculty: Faculty):
        pass

```

Figure 20: Coursemanager interface and implementation at CourseMaangerImp class

Figure 19: Course class with Coursemanager dependency injection at constructor with composite relationship

Course class enrollment limits and prerequisites validated at CourseMaangerImp as in the figure 21. Remove Students and assign faculty also need validations which is implemented in same implementation class.

```

class CourseManagerImp(ICourseManager):
    # Assign Student to Course
    def add_student(self, course, person_id: str, completed_courses: List[str]):
        course_code = course.course_code

        # check meets the prerequisites
        if course.should_have_completed is not None:
            for required in course.should_have_completed:
                if required not in completed_courses:
                    raise ValueError(f"Cannot enroll: {person_id} prerequisite {required} not completed for {course_code}")

        # check maximum allowed reached
        if len(course.reading_students) >= course.max_students_allowed:
            raise ValueError(f"Course {course_code} reached maximum allowed limit [{course.max_students_allowed}]")

        course.reading_students.append(person_id)
        Logger.info(f"Student {person_id} \t assigned to {course_code}")

```

Figure 21: Enrollment limits and prerequisites validation

2.6 Unit Testing and Loggers

Unit Tests has been implemented for covering course logics, student logics etc...

Test Class	Unit Test Name
TestCourseAddRemoveStudent	test_add_student_success_ok
	test_add_student_prerequisites_not_completed_raises_error_ok
	test_add_student_exceeds_max_allowed_raises_error_ok
	test_remove_student_ok
	test_remove_student_not_found_raises_error_ok
TestFacultyCourseAssignment	test_assign_faculty_to_course_success_ok

TestSecureStudentRecord	test_enroll_ok test_enroll_course_max_reached_and_throw_error_ok test_gpa_calculation_ok test_set_valid_gpa_ok test_set_invalid_gpa_range_throw_error_ok test_set_invalid_gpa_type_throw_error_ok test_change_student_ok
TestStudentEnrollment	test_enroll_new_course_ok test_enroll_existing_course_raise_error_ok
TestStudentDropCourse	test_drop_existing_course_ok test_drop_nonexist_course_raises_error_ok
TestCalculateGPA	test_calculate_gpa_with_grades_ok test_calculate_gpa_no_grades_raise_error_ok
TestStudentAcademicStatus	test_academic_status_deans_list_ok test_academic_status_good_ok test_academic_status_probation_ok
TestStudentSetResults	test_set_results_success_ok test_set_results_course_not_enrolled_raises_error test_set_duplicate_resut_raises_error

Table 1: Unit Tests

To run these tests execute below command, LOG_ENABLED=1 flag can be set to enable logs

```
PYTHONPATH=src python -m unittest discover -s tests
PYTHONPATH=src LOG_ENABLED=1 python -m unittest discover -s tests
```

Logger class is implemented for seperation and to have future proof code like to handle cloud logger in future.

```
class Logger:

    PREFIX = "LOG"
    enabled = os.environ.get("LOG_ENABLED", "0") == "1"

    @staticmethod
    def info(message: str):
        if Logger.enabled:
            print(f"[INFO] {Logger.PREFIX}: {message}")
```

Figure 21: Enrollment limits and prerequisites validation

3. Question 2: Data Analysis: E-commerce Data Analysis

3.1 Multi-Source Data Collection

3.1.1 Arcitecture

Webscraping/ and Data Saving implemented with Python clean architecture approach. Here domain and data layer has been seperated with identifying usecases, managers, components and model.

UseCases [domain/usecases]

Below are the identified uses cases

- 1) CollectDataUseCases
 - a) BookScrape
 - b) EcommerceScrape
 - c) RSSScrape
- 2) SaveDataUseCases
 - a) SaveJson

Managers [domain/manager]

These are interfaces only(ABC in python) Implementation of this managers goes in [data/manager]

- 1) IHttpClient
- 2) ISaveFile
- 3) IbookScrape
- 4) IECommerceScrape
- 5) IRSSXMLScrape

3.1.2 Dependency Injection

Since we use clean architecture, adding dependency injection would make the code more future-proof and allow mock injection for testing. Used python lib “dependency-injector==4.48.1”

Code goes in [di/container]

Object	Scope
httpClient	Singleton
save_data	Singleton
webScraper	Singleton
bookScraper	Factory
ecommerceScraper	Factory
rssScraper	Factory

Table 2: Dependency Injection Scopes

As depicted in Table 2 http client, data serializer, web scraper are reused and can be goes in singleton scope. Book Scraper, Ecommerce scraper and rss scraper would goes in videmodel so they are factory scope which created each time new instance when needed.

3.1.3 Async Requests

HTTP request processing is time consuming with standard processing of synchronized requests because it blocks the thread. So, two types of HTTP request implementations were created, both following the IHttpClient interface in figure 22. The dependency injector will

create the right client based on what the user needs during program execution, as shown in Figure 23

1. HttpClientSyncImp – thread blocking
2. HttpClientImp – Python Coroutine backed non blocking calls

```
class IHttpClient(ABC):
    @abstractmethod
    async def get(self, url) -> ApiResponse:
        pass
```

Figure 22: Http Client interface

```
use_async = os.environ.get("ASYNC", "1") == "1"

httpClient: providers.Provider[IHttpClient] = providers.Singleton(
    HttpClientImp if use_async else HttpClientSyncImp,
    timeout=3
)
```

Figure 23: Dependency Injector would return appropriate client type

3.1.4 Utils

Two utility classes were created: one for colorful console logs (info, warn, error, verbose) and another as a decorator for measuring execution time

BooksScarping

Goes under: data/manager/ book_scrape_imp.py

The data class BookDetails were use to keep per book scrape result as shown in figure 24. For the implementation all the records avaiable has been scraped and saved.

```
@dataclass
class BookDetail:
    title: str
    price: str
    category: str
    available: bool
    stock_count: int
    rating: str
    description: str
    upc: str
    product_type: str
    price_excl_tax: str
    price_incl_tax: str
    tax: str
    num_reviews: int
```

Figure 24: Book Details model class

For scraping and iteration two urls were used with repetition.

```
SCRAPE_HOST = https://books.toscraper.com
PAGE_URL_TEMPLATE = SCRAPE_HOST + "/catalogue/page-{}.html"
BOOK_URL_TEMPLATE = SCRAPE_HOST + "/catalogue/{}"
```

From PAGE_URL_TEMPLATE we scrape each page and from BOOK_URL_TEMPLATE we scrape each book. For page scraping used “article.product_pod” selector element and extracted the real url of the book page. And after using BOOK_URL_TEMPLATE and extracted book url scraped the book details. Here extracting the book category also important needed and used the seletor eleement “ul.breadcrumb li a” here.

Used python coroutines for each page scrape and book scrape with introducing decent delays for not to overwhelming the server as shown in Figures 25 and 26.

```

async def _fetchPage(self, pageNumber: int):
    pageUrl = PAGE_URL_TEMPLATE.format(pageNumber)
    await asyncio.sleep(random.uniform(1.5, 8.8))
    response = await self.client.get(pageUrl)

    if not response.ok(response):
        return []

    booksMetaList = self.scrafer.getFromHtml(response.body, "article.product_pod", _mapBookMeta)

    bookScrapeTasks = []
    for bookMeta in booksMetaList:
        bookScrapeTasks.append(asyncio.create_task(self._fetchBook(bookMeta=bookMeta)))

```

Figure 25: Coroutine for each book scrfape with random delay (1.5, 8.8) sec

```

async def _fetchBook(self, bookMeta: BookMeta) -> BookDetail | None:
    await asyncio.sleep(random.uniform(2.1, 5.6))
    bookUrl = BOOK_URL_TEMPLATE.format(bookMeta.url)
    response = await self.client.get(bookUrl)

    if not response.ok(response):
        return None

    bookDetail = self.scrafer.getFromHtmlSingle(response.body, _mapBook)

    return bookDetail

async def collectData(self, pageCount: int) -> List[BookDetail]:
    pageScrapeTasks = []
    for i in range(1, pageCount + 1):
        pageScrapeTasks.append(asyncio.create_task(self._fetchPage(i)))

```

Figure 26: Coroutine for each page scrfape with random delay (2.1, 5.6) sec

If a http request fails, retry with exponential backoff mechanism has been implemented. This been improved with random jitter which would avoid another serverf spike if multiple request fails together. This is shown in figure 27. If permanantly fails request return with code = -1

```

async def _retry_async(self, url: str):
    attempt = 1
    while attempt < self.maxRetries:
        try:
            response = await self._request(url)

            if(attempt>1):
                Logger.verbose(f"Success! After failed, Attempt: {attempt}/{self.maxRetries}, url:{url}")
            else:
                Logger.verbose(f"Success! url:{url}")

            return response
        except Exception as e:
            attempt += 1
            slepTime = min(self.backOff * (2 ** (attempt - 1)), 2.0)
            Logger.warn(f"Request Failed!, Attempt: {attempt}/{self.maxRetries} after {slepTime} secs, url: [{url}]")
            await asyncio.sleep(slepTime)

    Logger.error(f"Permanent Failure, url:{url}")
    return ApiResponse(status_code=-1, body=f"Failed with max retry: {self.maxRetries}")

```

Figure 26: Exponential back-off with random jitter

Performance Improvement

Coroutines with async http request has tremendosuly improved the time taken to Bookscraping of website books.toscrape.com and extract all 1000 books as depicted in table 3

Request Type for (books.toscrape.com)	Time Taken for 1000 books to srcape
Sync	500 + seconds
Async with descent delay	~17 seconds

Table 3: Time matrix betwwne async request and synced request

Data Saving

Extracted Data has been saved as json in the directory “data_collection/raw_files/books.json”. This has been added to git fot the reference.

```
[
  {
    "title": "A Light in the Attic",
    "price": "£51.77",
    "category": "Poetry",
    "available": true,
    "stock_count": 22,
    "rating": "Three",
    "description": "It's hard to imagine a wor",
    "upc": "a897fe39b1053632",
    "product_type": "Books",
    "price_excl_tax": "£51.77",
    "price_incl_tax": "£51.77",
    "tax": "£0.00",
    "num_reviews": 0
  },
]
```

Figure 27: Books.json

EcommerceScraping

This is exactly similar to Bookscraping implementation and reused all codes except the html element scraping.

Goes under: data/manager/ecommerce_scrape_imp.py

The data class ProductDetail were use to keep per book scrape result as shown in figure 24. For the implementation all the records available has been scraped and saved.

```
@dataclass
class ProductDetail:
    title: str
    price: str
    description: str
    stock: str
    sku: str
    categories: list[str]
    tags: list[str]
```

Figure 28: ProductDetail model class

For scraping and iteration `PAGE_URL_TEMPLATE` url were used with repetition in page level. It gives the url to the product which used for scraping the product details.

```
SCRAPE_HOST = "https://scrapeme.live/shop"
PAGE_URL_TEMPLATE = SCRAPE_HOST + "/page/{}/"
```

Data Saving

Extracted Data has been saved as json in the directory

“data_collection/raw_files/products.json”. This has been added to git fot the reference.

```
[
  {
    "title": "Pidgeot",
    "price": "£185.00",
    "description": "This Pokémon has a dazzling plumage of beautifully glossy",
    "stock": "90 in stock",
    "sku": "8462",
    "categories": [
      "Bird",
      "Pokemon"
    ],
    "tags": [
      "Bird",
      "Keen Eye",
      "pidgeot"
    ]
  },
]
```

Figure 29: products.json

RSSXMLScraping

RSS_HOST = "https://rss.nytimes.com/services/xml/rss/nyt/HomePage.xml"

RSS Element Data class in figure 30

```
@dataclass
class RSSElement:
    title: str
    link: str
    description: str
    pubDate: str
```

Data Saving

Extracted Data has been saved as json in the directory "data_collection/raw_files/news_rss.json". This has been added to git for the reference.

```
> PAGE_URL_TEMPLA
{
  "title": "Jimmy Kimmel's Show to Return to ABC on Tuesday Night",
  "link": "https://www.nytimes.com/2025/09/22/business/media/jimmy-kimmel-r",
  "description": "The network's removal of Mr. Kimmel's show last week almos",
  "pubDate": "Tue, 23 Sep 2025 00:18:55 +0000"
},
{
  "title": "Politicians and Celebrities Weigh In on Jimmy Kimmel's Return to",
  "link": "https://www.nytimes.com/2025/09/22/us/jimmy-kimmel-show-return-r",
  "description": "Some celebrated ABC's decision to rescind the show's susp",
  "pubDate": "Mon, 22 Sep 2025 23:49:29 +0000"
},
]
```

Figure 30: news_rss.json

3.2 Data Cleaning and Preprocessing

Implementation at : data_processing/ books_processing.ipynb

3.2.1 Loading, print and describe the data frame

Here we used the “pandas” library. First the json file was loaded into a data frame and then the head was printed to show the first five rows as shown in figure 31. After we describe the dataframe as shown in figure 32.

Print Data Frame Head

```
df.head()
```

	title	price	category	available	stock_count	rating	description	upc	product_type	price_excl_tax	price_incl_tax	tax	num_reviews
0	A Light in the Attic	£51.77	Poetry	True	22	Three	It's hard to imagine a world without A Light I...	a897e39b1053632	Books	£51.77	£51.77	£0.00	0
1	Tipping the Velvet	£53.74	Historical Fiction	True	20	One	"Erotic and absorbing...Written with startling...	90fa61229261140a	Books	£53.74	£53.74	£0.00	0
2	Soumission	£50.10	Fiction	True	20	One	Dans une France assez proche de la nôtre, un h...	695744c3847a760	Books	£50.10	£50.10	£0.00	0
3	Sharp Objects	£47.82	Mystery	True	20	Four	WICKED above her hipbone, GIRL across her hear...	e00eb4fd7b871a48	Books	£47.82	£47.82	£0.00	0
4	Sapiens: A Brief History of Humankind	£54.23	History	True	20	Five	From a renowned historian comes a groundbreak...	4165285e1663650f	Books	£54.23	£54.23	£0.00	0

Figure 31: Books Data Frame Head

Statistical Summary

```
df.describe()
```

	stock_count	num_reviews
count	1000.000000	1000.0
mean	8.585000	0.0
std	5.654622	0.0
min	1.000000	0.0
25%	3.000000	0.0
50%	7.000000	0.0
75%	14.000000	0.0
max	22.000000	0.0

Figure 32: Data Frame statistical summary

3.2.2 Remove Special Characters and Unnessasary columns

At a glance we can see the special character “£” need to be removed from the price columns and the upc column does not have any significant effect on the analysis, so it should be dropped. And no reviews are avaialable as depicted in figure 32. “num_reviews” column also we can drop.

After when we describe the dataframe it shows statistics as in figure 33.

	price	stock_count	price_excl_tax	price_incl_tax	tax
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.0
mean	35.07035	8.585000	35.07035	35.07035	0.0
std	14.44669	5.654622	14.44669	14.44669	0.0
min	10.00000	1.000000	10.00000	10.00000	0.0
25%	22.10750	3.000000	22.10750	22.10750	0.0
50%	35.98000	7.000000	35.98000	35.98000	0.0
75%	47.45750	14.000000	47.45750	47.45750	0.0
max	59.99000	22.000000	59.99000	59.99000	0.0

Figure 32: cleaned price data frame

By looking at figure 32 we can conclude tax is also not applicable and we can drop columns [price_excl_tax, price_incl_tax, tax].

3.2.3 Text Normalization

“Description” column

1. “more...” this could be removed
2. White spaces in front and back can be removed
3. Tabs, newlines, multiple spaces can be shrink down to single space.

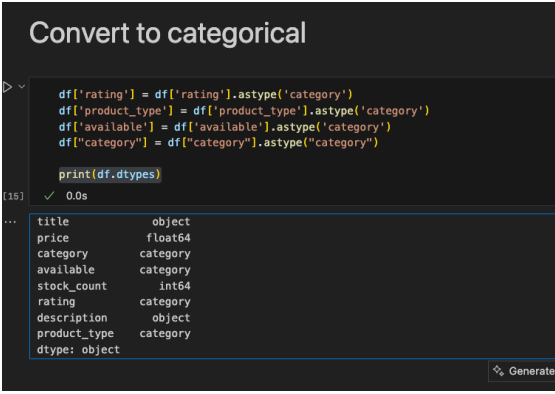
“Title” column

Texts inside the parenthesis we can remove.

3.2.4 Convert to categorical

The following columns have been identified as categorical and shows in figure 33 when displayed the types.

- 1) Rating
- 2) Product Type
- 3) Available
- 4) Category



```
df['rating'] = df['rating'].astype('category')
df['product_type'] = df['product_type'].astype('category')
df['available'] = df['available'].astype('category')
df['category'] = df['category'].astype('category')

print(df.dtypes)
```

[15] ✓ 0.0s

title	object
price	float64
category	category
available	category
stock_count	int64
rating	category
description	object
product_type	category
dtype: object	

Generate

Figure 33: Convert to categorical

3.2.5 Categorical Data cleaning

Now we can check how many unique categories exist in the data. There should be 50 as in the original Books Scrape website.

```
num_categories = df["category"].nunique()
print(num_categories) # prints 50 so we are fine
```

Validity of Type Books and Available

```
non_books_count = (df['product_type'] != 'Books').sum()
unavailable_books = (df['available'] == False).sum()
print(f"Non books:{non_books_count}, unavaialable:{unavailable_books}")
```

```
Non books:0, unavaialable:0
```

Conclusion: The Books and Available columns do not have a significant effect on the statistical analysis. So it's good to drop these two columns too.

Ratings currently in text format and better to assign int value for rating as it is easy for processing and storing.

Final Data frame we export to json lines

(data_processing/processed_files/books.processed.jsonl) as its is easier to read and processing larger files as in figure 34.

```
{
  "title": "A Light in the Attic",
  "price": 51.77,
  "category": "Poetry",
  "stock_count": 22,
  "rating": 3,
  "description": "It's hard to imagine a world without A Light in the Attic"
},
{
  "title": "Tipping the Velvet",
  "price": 53.74,
  "category": "Historical Fiction",
  "stock_count": 20,
  "rating": 1,
  "description": "\\"Erotic and absorbing...Written with startling"
},
{
  "title": "Soumission",
  "price": 50.1,
  "category": "Fiction",
  "stock_count": 20,
  "rating": 1,
  "description": "Dans une France assez proche de la n\u00f4tre, un homme s\u2019eng"
},
{
  "title": "Sharp Objects",
  "price": 47.82,
  "category": "Mystery",
  "stock_count": 20,
  "rating": 4,
  "description": "WICKED above her hipbone, GIRL across her heart Words are like"
},
{
  "title": "Sapiens: A Brief History of Humankind",
  "price": 54.23,
  "category": "History",
  "stock_count": 20,
  "rating": 5,
  "description": "From a renowned historian comes a grou"
},
{
  "title": "The Requiem Red",
  "price": 22.65,
  "category": "Young Adult",
  "stock_count": 19,
  "rating": 1,
  "description": "Patient Twenty-nine.A monster roams the halls of Soothing"
},
{
  "title": "The Dirty Little Secrets of Getting Your Dream Job",
  "price": 33.34,
  "category": "Business",
  "stock_count": 19,
  "rating": 4,
  "description": "Drawing on his extensive"
},
{
  "title": "The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull",
  "price": 17.93,
  "category": "Default",
  "stock_count": 19,
  "rating": 3,
  "de"
}
```

Figure 34: Cleaned books exported to json lines file

3.3 Advanced Statistical Analysis

Can find at : [anaysis/books_analysis.ipynb]

3.3.1 Price distribution analysis across categories

Here we need to get the summary of book prices for each category. And based on the results following has been observed

```
df.groupby("category")["price"].describe()
```

1. Default category has 152 books, Fiction category has 65, some categories only have 1 book (Novels, Crime)
2. Based on average price per category its noticeable that Health books are expensive(51.45 mean) and Crime books are cheaper(10.97 mean). Most categories falls between 30 to 40
3. Based on standard deviation its noticeable that Autobiography has large price variation (std: 21.47) and Health are consistant(std: 2.45)
4. Some categories have only one book so standard deviation is NAN because to calculate std needs at least 2 values.

3.3.2 Rating patterns and statistical summaries

Rating distribution is little skewed toward lower ratings but noticeable that it spread out fairly.

1 – 226
2 – 196
3 – 203
4 – 179
5 – 196

When we look at Average rating per category Adult Fiction as highest rating (5.0) and crime has lowest rating which is 1.0

3.3.3 Correlation analysis (price, rating, availability)

Availability column we dropped already because all are available.

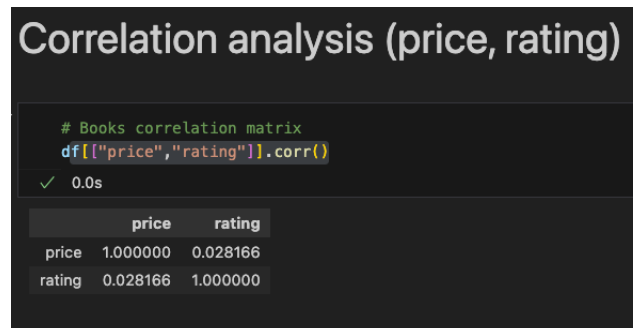


Figure 35: Correlation analysis

Based on the value (close to 0) we can tell knowing a book price gives no information about its rating, and vice versa. So uncorrelated.

3.3.4 Category popularity analysis

After checking how many books are in each category as in figure it can observe that Default, Nonfiction, Sequential Art, Fiction, Young Adult contains more than 50 books and nearly 50% of categories contain less than 10 books.

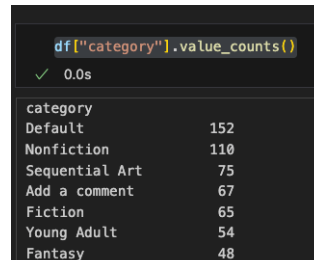


Figure 36: Category popularity analysis

Category ratings distribution

Most categories have average ratings which is 2–3, Generally low to moderate rating.

Outliers:

1. Very few books < 5
2. Average rating is way far away from global mean
3. Ratings in the category is highly variable (std > 2)

15 categories fulfill above as in figure 37.

category	count	mean	std	min	25%	50%	75%	max
Academic	1.0	2.000000	NaN	2.0	2.00	2.0	2.0	2.0
Adult Fiction	1.0	5.000000	NaN	5.0	5.00	5.0	5.0	5.0
Christian	3.0	2.666667	2.081666	1.0	1.50	2.0	3.5	5.0
Contemporary	3.0	2.333333	2.309401	1.0	1.00	1.0	3.0	5.0
Crime	1.0	1.000000	NaN	1.0	1.00	1.0	1.0	1.0
Cultural	1.0	1.000000	NaN	1.0	1.00	1.0	1.0	1.0
Erotica	1.0	5.000000	NaN	5.0	5.00	5.0	5.0	5.0
Health	4.0	3.750000	1.500000	2.0	2.75	4.0	5.0	5.0
Historical	2.0	3.000000	2.828427	1.0	2.00	3.0	4.0	5.0
Novels	1.0	5.000000	NaN	5.0	5.00	5.0	5.0	5.0
Paranormal	1.0	1.000000	NaN	1.0	1.00	1.0	1.0	1.0
Parenting	1.0	2.000000	NaN	2.0	2.00	2.0	2.0	2.0
Politics	3.0	2.333333	1.527525	1.0	1.50	2.0	3.0	4.0
Short Stories	1.0	1.000000	NaN	1.0	1.00	1.0	1.0	1.0
Suspense	1.0	3.000000	NaN	3.0	3.00	3.0	3.0	3.0

Figure 37: outliers

3.4 Visualization

Can find at : [visualizations/books_visalization.ipynb]

3.5 Predictive Analysis

Can find at : [predictive_analysis/books_predictive_analysis.ipynb]

4. AI Ethics in Healthcare Data

4.1 Healthcare Data Privacy Challenges

Healthcare data is categorized under sensitive data which needs a greater layer of protection when it comes to data security. Healthcare data may not just contain medical records but also it may include genomic data like DNA, sensor recorded data like (heart rates and sleep patterns). Unauthorized access to this sensitive data could cause far more damage than financial. For example it could lead to discrimination and other social level consequences.

In the EU emphasizes consent and the right to be forgotten, HIPAA[1] in the US enforces stricter requirements for data collection, storage and sharing. That includes additional security measurements like encryption, access control and audit logs, which are mandatory.

In order to anonymizing medical data removing precise identifiers like names, identification numbers are not sufficient. Using data combinations the patient may be identified again easily. When it comes to AI friendly data maintaining anonymization could be challenging.

It is the duty of the researchers to balance the research and their privacy.

Eg: Training a skin cancer detection model from actual patient images. This requires a large dataset of images showing cancerous skin as well as spots without it. To obtain all the cancer images needed real world data must be accessed that will contain patients records. So researchers must always remember that they should only extract the data they need. Such as just Image with Cancer/not classification. No patient identification data may ever be extracted, Saved or shared.

4.2 Algorithmic Bias in Medical AI

Health disparities come from algorithmic biased medical AI becoming an increasing concern. So it's equally important to find where these doubts are coming from and try to avoid them from going forward down the future medical AI. Indicators of bias include

- a) Demographic bias: This could be over representation of people of particular ages, genders or ethnic groups. For example for above skin cancer model creation 80% of images come from male patients age over 50 and small portion comes from female patients and male patients below 50

- b) Geographical bias: May not apply globally to all regions because the data is from particular areas or sites only. For example for above skin cancer model creation 80% of images comes from USA and rest from other parts of the world.
- c) Socioeconomic bias: Data from poor communities can also present predictions that are uneven across some parts of society. For example for skin cancer model we get 80% of data from a hospitals where visited by homeless people who expose to sunlight very often than rich communities.

To solve this problem measures of fairness such as equal opportunity, group fairness, and predictive equality can be utilized to ensure fairness. As a implementation an audit various groups can be imposed when collecting data.

4.3 Ethical Decision-Making Framework

Checklist to creating ethical framework to insure responsible AI in Health.

- 1) Must get patient consent and communicate clearly their data would be used within AI system
- 2) Ensure data is anonymized and clearly communicate that to patients
- 3) Ensure data is secured
- 4) Audit bias in dataset and models
- 5) Measure any potential harm to patients before they use for AI
- 6) AI decisions explained to both patients and healthcare providers which improves the trust and let doctors confirm the advice given by the AI
- 7) Predictive model may reveal future risks which is unforeseen and this must be carefully communicate which does not trigger unnecessary alarms.

4.4 Stakeholder Impact Analysis

AI affects various stakeholders differently. Patients would benefit from faster diagnosis and customized treatment. For example if skin cancer could detect using ai model just with images they does not need to go through PET[2] scan which eliminates the medical risks associated with radioactive materials used in PET scan. But this also involves risks like false detection/Non detection.

Healthcare providers can use AI to aid clinical decision making. But they need be trained in interpreting outputs of AI systems and for maintaining the trust which patients place in them.

Research workers benefit from large datasets driving innovation. But they must ensure that data is used ethically and does no harm.

AI can cut the costs of healthcare from an economic/financial point of view, but from a social perspective It can make inequalities if marginalized groups are not concerned.

A balanced solution lies in using international standards for privacy and fairness which promote fairness in global health equity. This could include

- a) Using of wide range of audited data

- b) Involving healthcare workers, ethicists and data scientists in development and operations of health care systems.

References

[1] **Edemekong, P. F. (2024).** *Health Insurance Portability and Accountability Act (HIPAA) Overview*. StatPearls Publishing. Retrieved from

[2] Phelps, Michael E. *PET: Molecular Imaging and Its Biological Applications*. Springer, 2004.